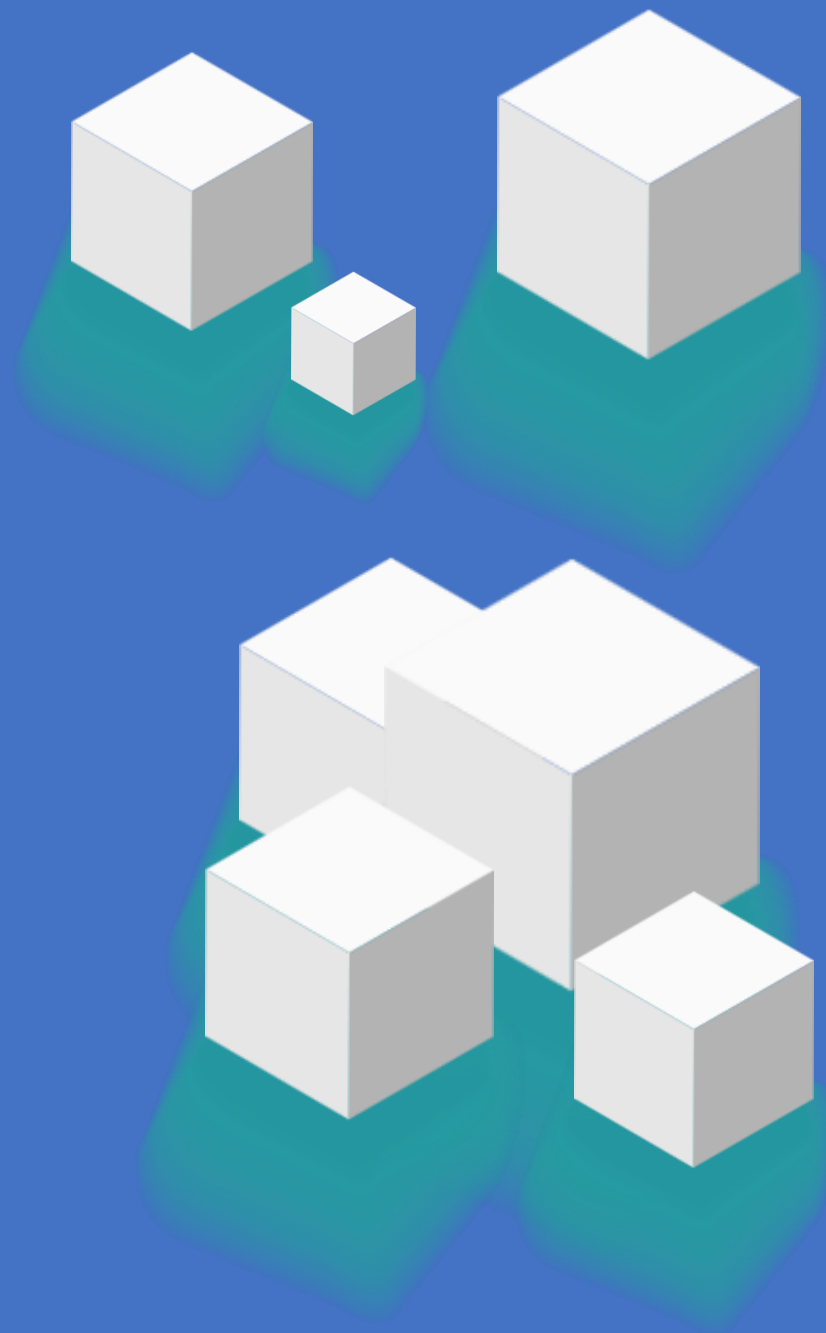
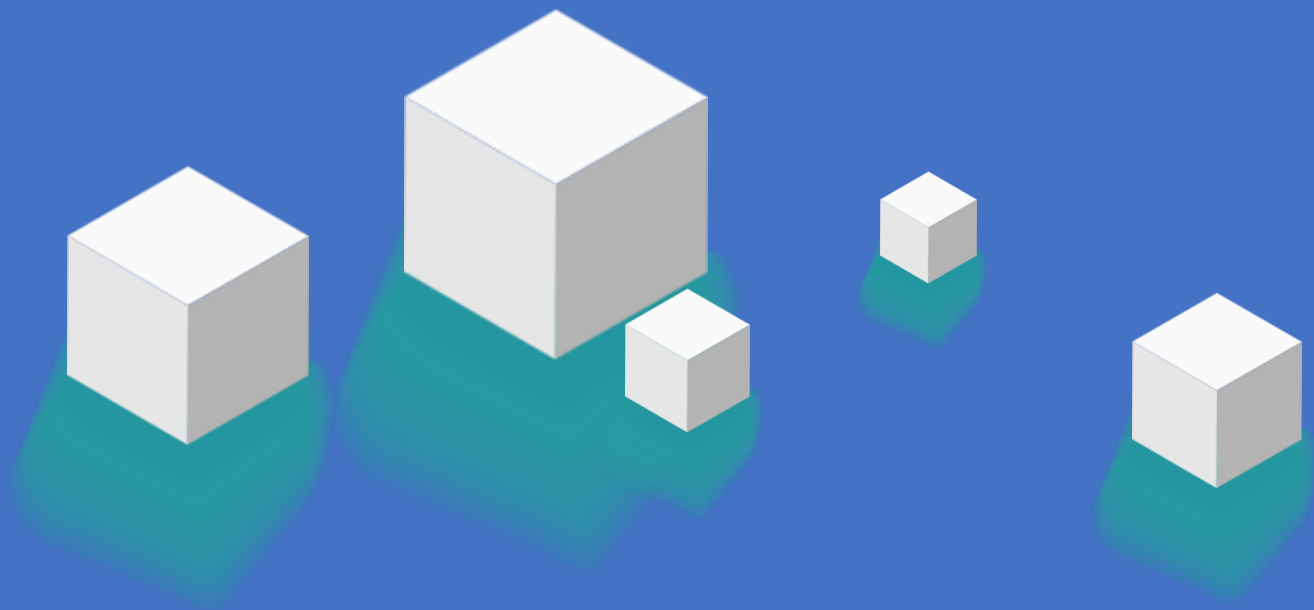


投资晨会



>> 今天的学习目标

投资晨会

- 投资晨会场景
- CASE：多因子选股

多因子选股的pipeline、因子库

预处理：三件套、因子合成的方法

分层回测 + 单因子 IC 诊断

- CASE：板块数据准备
- CASE：板块轮动分析

3层信号体系、5 个核心导数指标、拐点判定逻辑

一年滚动回测、事件规律挖掘

- CASE：晨会 workflow

LangGraph使用、架构设计、State设计

投资晨会场景

投资晨会场景

每天早上开盘前, 用一套自动化流水线, 在 30 分钟内拿到: 今天该关注哪些板块、哪些股票

=> 9:00 cron 触发 → 9:20 拿到一份带板块 + 选股 + 操作建议的晨报

从单标的判断 => 扩展到 全市场筛选

之前我们的策略输入大多是已经选好的标的, 系统只负责对单只股票做信号判断和下单。

这种模式在 A 股 5000+ 只股票的池子里有个明显缺口: 谁来告诉系统: 今天值得关注哪几只?

- 选股: 板块轮动锁定强势行业 + 多因子打分挑出 Top N 个股
- 信号: 对选出的标的逐个评估买卖时机, 输出操作建议

因子投资

Thinking: 什么是因子投资?

一只股票未来涨多少, 不是完全随机, 而是可以由一组特征 (因子) 部分解释。

常见的有:

- 价值因子 (PE / PB 低的股票长期跑赢)
- 规模因子 (小盘股长期跑赢大盘股)
- 动量因子 (近期涨得多的短期还会涨)
- 质量因子 (ROE 高、负债低的公司更稳)。

因子投资

因子投资思想是过去 60 年学术界一步步构建出来的：

年代	里程碑	新加入的因子
1964	CAPM	市场 beta (单因子, 一切收益都用市场涨跌解释)
1992	Fama-French 3 因子	+ 规模 + 价值 (小盘股、低估值股长期超额)
1997	Carhart 4 因子	+ 动量 (近期强者恒强)
2014	Fama-French 5 因子	+ 盈利 + 投资 (赚钱能力强、再投资保守的公司更好)
当代	Barra 风险模型	10+ 风险因子, 含行业、规模、价值、质量、流动性等

IC 与 IR：判断一个因子靠不靠谱

Thinking: 选了一堆因子, 怎么知道哪个真的能赚钱、哪个只是看起来唬人?

- IC: 看因子打的分数和未来涨跌方向是否一致

IC (Information Coefficient) = $\text{corr}(\text{今天的因子值}, \text{未来 } N \text{ 天的股票收益})$

=> 横截面相关系数。

今天给所有股票按这个因子打分, 然后看 N 天后的实际涨跌, 两者方向越一致, IC 越高。

比如: 因子是"过去 20 日涨幅", 如果"涨幅大的股票"在未来 5 天平均也涨得多, IC 就是正的。

- IR: 看这种一致性是不是长期稳定

IR (Information Ratio) = $\text{mean(IC)} / \text{std(IC)}$

=> 把每天 (或每周) 算出来的 IC 序列再算 均值 / 标准差, 衡量这个因子是偶尔灵还是长期都灵。

IR 高 = 可以重仓押。

IC 与 IR：判断一个因子靠不靠谱

IC / IR 的经验区间 (单技术因子, A 股):

IC 量级	含义	典型场景
0.01 - 0.03	弱因子	大部分单技术因子
0.03 - 0.06	中等因子	多因子合成后的 alpha
0.06+	强因子	少见, 被发现就慢慢失效 的稀缺品

IR 量级	含义
< 0.2	不稳定, 大概率是噪音
0.2 - 0.5	有信号但波动大, 可作辅助
0.5 - 1.0	优秀, 业内主力因子的水平
> 1.0	顶级, 极少见, 出现了通常很快被人挖走

板块轮动的思路

Thinking：今天该看哪些板块这个问题, 思路都有哪些？

- 路径一: 自上而下 -- 美林时钟 (宏观驱动)

美林时钟 (2004 Merrill Lynch): 把宏观经济划成 4 个阶段, 每个阶段不同的资产 / 行业受益。

逻辑是先判断 现在经济在哪个象限, 再推 应该买哪类板块:

阶段	经济特征	受益资产	对应 A 股板块
复苏	低通胀 + 高增长	股票, 周期股	金融 / 地产 / 周期
过热	高通胀 + 高增长	大宗商品	有色 / 煤炭 / 钢铁
滞胀	高通胀 + 低增长	现金	避险, 防御消费
衰退	低通胀 + 低增长	债券	医药 / 消费 / 公用事业

美林时钟在 A 股的局限: 部分有效。

A 股的板块轮动很大程度上由政策催化 (产业政策、补贴、减税) 和主题爆点 (AI 算力、新能源、消费电子) 驱动, 跟宏观周期不完全同步。完全照搬美林时钟, 可能会经济过热应该买周期 -- 但实际市场已经在炒 AI。

板块轮动的思路

- 路径二: 自下而上 -- 价格行为驱动

不去预测宏观周期, 直接观察板块指数的价格本身在说什么:

哪个板块在加速上涨? 哪个板块开始钝化? 哪个板块跌幅在收敛 (可能见底)?

=> 这套思路对政策 / 主题驱动的 A 股更适用, 也更容易工程化。

TO DO 三层信号体系:

- 静态强度 (零阶): 短期动量 ($_21$) + 相对沪深 300 强度 (RS_60) + 量能配合 (VOL_RATIO), z-score 合成出现在哪个板块强
- 速度 (一阶导): ROC_20 + MA20 斜率, 看还在加速 or 已经放缓
- 加速度 (二阶导): MACD 柱体 + 斜率变化, 把板块状态分成 【主升加速 / 高位钝化 / 主跌 / 左侧抄底】
4 个象限

项目结构

投资晨会-人机协作/

└─ CASE-A-板块数据准备/

数据基础设施

- | └─ db_config.py MySQL 连接 + 通用 SQL 工具
- | └─ industry_meta.py 申万一级/二级元数据 -> trade_stock_status.sector_1/2
- | └─ stock_kline_loader.py 个股 K 线 -> trade_stock_daily
- | └─ sector_index_builder.py 板块每日聚合 + 等权合成 OHLC -> trade_sector_daily
- | └─ run_init.py 一键全量初始化 (建表 + 拉数据)
- | └─ run_daily.py 每日增量 (8:30 cron)
- | └─ sql/schema.sql 3 张表结构 (后续 CASE 共用)

└─ CASE-B-板块轮动分析/

信号挖掘

- | └─ sector_loader.py 从 trade_sector_daily 加载板块指数
- | └─ industry_strength.py 静态强度 ($_21 + RS_60 + VOL_RATIO$)
- | └─ derivatives.py 一阶导 (ROC, MA20_SLOPE) + 二阶导 (MACD_HIST, MA20_ACCEL)
- | └─ inflection_detector.py 4 象限拐点分类 (accel_up / decel_up / accel_down / decel_down)
- | └─ backtest.py 一年滚动回测 (落 rotation_events.csv, 11K+ 事件)
- | └─ rotation_insights.py 零依赖深挖器 (切换矩阵 / 衰减曲线 / 板块可靠度 / 月度温度计)
- | └─ run_today.py 当日综合视图 (静态 + 拐点)

项目结构

投资晨会-人机协作/

└─ CASE-C-多因子选股/

选股

- └─ stock_pool.py 指数成分股 + 行业归类 + 可交易过滤
- └─ factor_lib.py 10 个因子的计算
- └─ preprocessor.py Winsorize / Z-score / 行业中性化
- └─ synthesizer.py 等权 / IC 加权 / Lasso 三种合成
- └─ layered_backtest.py 5 层回测 + 单因子 IC 诊断
- |

└─ CASE-D-投资晨会 workflow/

编排 + 推送

- └─ graph.py LangGraph 4 节点 DAG
- └─ pusher.py 钉钉 / 企业微信 / console
- └─ lib/ 从 CASE-B / CASE-C 复制的内嵌副本
 - └─ db_config.py
 - └─ rotation_runner.py CASE-B 精简版
 - └─ factor_runner.py CASE-C 精简版
 - └─ outputs/reports/ 晨报 .md / .html 落盘

CASE: 多因子选股

CASE：多因子选股

Thinking：之前都做过哪些？

多因子选股的方法方法：

因子计算 → 8 因子等权打分 → Top-N 月度调仓

=> 对比过纯小市值 / 增强小市值 / ML 增强

多因子选股的pipeline

多因子选股是量化投资最经典也最有效的策略之一。它的逻辑很简单:

- 构造一组因子 (能解释股票收益的特征)
- 对每只股票算因子值, 加权合成=>alpha 分数
- 按 alpha 分数排序, 选 Top N 持有, 定期换仓

多因子选股的pipeline

Thinking: 多因子选股工程上有哪些常见的坑?

- 因子量纲差很多 (动量 0.01 级 vs 流动性 1e9 级)

=> 解决方法: Z-score 标准化

- 个别极端值主导整个因子分布

=> 解决方法: MAD 去极值

- 低 PE 选出来全是银行

=> 解决方法: 行业中性化

- 因子可能反向有效 (动量 vs 反转)

=> 解决方法: 单因子 IC 诊断

- 因子失效不知道

=> 解决方法: 分层回测 + IR 监控

多因子选股的pipeline

完整的Pipeline:

- Step1, 构建因子库 => 10 因子计算
- Step2, 预处理 => 去极值/Z-score/中性化
- Step3, 合成 => 等权 / IC 加权
- Step4, 分层回测 => 5 层 + 多空
- Step5, IC 诊断 => 单因子有效性

因子库

10 个因子分 5 类

类别	因子名	计算	金融含义
动量	_1M	过去 21 日累计收益	短期动量
	_3M	过去 63 日累计收益	中期动量
	_6M	过去 126 日累计收益	长期动量
反转	REV_5D	$-1 \times$ 过去 5 日收益	短期反转 (取负号)
波动率	VOL_20	$-1 \times$ 20 日年化波动	低波动好 (取负号)
	VOL_60	$-1 \times$ 60 日年化波动	同上
流动性	LIQ_20	$-1 \times \log(20 \text{ 日均成交额})$	过度被关注的票反而 alpha 低
	TURN_20	$-1 \times$ 20 日均换手率	低换手好
技术	RSI_14	$RSI(14) - 50$	动能指标
	BIAS_20	$-1 \times (\text{close} - \text{MA20}) / \text{MA20}$	乖离过大易回调 (取负号)

因子库

```
def calc_factors_for_one(df: pd.DataFrame) -> dict:
    # 给定单只股票的日 K 线 DataFrame, 算 10 个因子
    close = df["close"].astype(float)
    returns = close.pct_change().dropna()
    factors = {}

    # 动量
    factors["_1M"] = close.iloc[-1] / close.iloc[-22] - 1
    factors["_3M"] = close.iloc[-1] / close.iloc[-64] - 1
    factors["_6M"] = close.iloc[-1] / close.iloc[-127] - 1

    # 反转 (取负号 -- 短期涨多易跌)
    factors["REV_5D"] = -(close.iloc[-1] / close.iloc[-6] - 1)

    # 波动率 (年化, 取负号 -- 低波动好)
    factors["VOL_20"] = -(returns.tail(20).std() * sqrt(250))
    factors["VOL_60"] = -(returns.tail(60).std() * sqrt(250))
```

```
# 流动性 (取对数后取负号)
liq_20 = df["amount"].tail(20).mean()
factors["LIQ_20"] = -log(max(liq_20, 1.0))

# RSI 14 + 乖离
rsi = _calc_rsi(close, 14)
factors["RSI_14"] = rsi - 50
ma20 = close.rolling(20).mean().iloc[-1]
factors["BIAS_20"] = -(close.iloc[-1] - ma20) / ma20

return factors
```

每个因子都是越大越好的方向。比如 VOL/LIQ/BIAS 这种越小越好的指标, 通过取负号统一成越大越好。

这样后面合成时直接相加就行, 不用记每个因子的方向。

[factor_lib.py](#)

因子库

运行结果 python factor_lib.py

	_1M	_3M	_6M	REV_5D	VOL_20	VOL_60	LIQ_20	TURN_20	RSI_14	BIAS_20	
600519.SH	-0.0350	-0.0069	0.0007	0.0268	-0.1793	-0.2381	-22.2715	-0.6963	-2.2499	0.0185	
000001.SZ	0.0036	-0.0309	-0.0129	0.0056	-0.1757	-0.1205	-20.6214	-0.9665	-0.4065	0.0014	
002594.SZ	0.0199	0.0676	-0.0361	-0.0201	-0.3350	-0.3252	-22.6560	-1.2116	-3.0449	-0.0039	
300750.SZ	0.1014	0.1957	0.1573	-0.0656	-0.4118	-0.3405	-23.3850	-1.0245	11.5404	-0.0800	
513100.SH	0.0576	-0.0058	0.0172	-0.0459	-0.3586	-0.2501	-19.9898	-1.2018	21.8009	-0.0710	

可以看出:

- 宁德时代 6 月动量 +15.7% 最强, RSI 11.5 强势;
- 茅台 1 月动量 -3.5% 弱, RSI 接近中性。

预处理：去极值

1. 去极值 (Winsorize) -- MAD 方法

`median = series.median()`

`mad = (series - median).abs().median()`

`[lower, upper] = median  $\pm$  n  $\times$  1.4826  $\times$  mad (默认 n=3)`

把超出区间的值截断到边界

1.4826 是高斯分布 MAD $\rightarrow$ std 的换算系数, 让 $N \times \text{MAD}$ 跟 $N \times \sigma$ 等价。

标准正态分布下, $\text{MAD} = 0.6745\sigma$, 因此 $\sigma = \text{MAD} / 0.6745 \approx 1.4826 \times \text{MAD}$ 。

乘以 1.4826 后, $n \times \text{MAD}$ 的截断范围与常用的 $n \times \sigma$ 等价, 保证概率尺度一致。

预处理：标准化

2. 标准化 (Z-score)

$$z = (x - \text{mean}) / \text{std}$$

把所有因子拉到 mean=0 / std=1, 不同因子可加可比。

预处理：行业中性化

3. 行业中性化 -- 关键中的关键

问题: 银行股 PE 普遍低 (5-10), 科技股 PE 普遍高 (30-100)。

如果不中性化, 选低 PE 会变成全选银行 -- 这是行业 beta, 不是真正的 alpha。

=> 每个因子在每个行业内单独做 Z-score, 然后再做一次全市场 Z-score。

```
def industry_neutralize(factor_series, industry_map):  
    df = pd.DataFrame({"factor": factor_series, "industry": pd.Series(industry_map)})  
    # 对每个行业组单独 Z-score  
    return df.groupby("industry")["factor"].transform(zscore)
```

预处理：三件套

实测效果 (模拟数据 + 100 只股票, 含极端值 5.0 和 -3.0)

[原始] mean=0.058, std=0.591, min=-3.0, max=5.0 <- 被极端值拉爆

[去极值后] mean=0.039, std=0.096, min=-0.196, max=0.266 <- 正常分布

[标准化后] mean=0.000, std=1.000, min=-2.45, max=2.36 <- $N(0,1)$

[中性化后] mean=0.000, std=1.000, min=-2.44, max=2.84 <- 行业内可比

如果不做预处理直接合成 => 100% 会被极端值或量纲主导, 因子完全无效。

行业中性化这一步通常能让 IC 从 0.02 提升到 0.05+。

因子合成的方法

方式 1: 等权合成

$\alpha = \text{mean}(\text{factor1}, \text{factor2}, \dots, \text{factor10})$

最简单, 假设所有因子同等重要。

=> 问题: 反向有效的因子会互相抵消。

方式 2: IC 加权 (推荐)

$\text{weight} = \text{IC} / \sum |\text{IC}|$

$\alpha = \sum (\text{weight} \times \text{factor})$

IC 越高的因子权重越大, 自动给有效因子加分 => 工业界主力方案。

因子合成的方法

方式 3: Lasso 回归

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=0.01)
model.fit(X_history, y_history_returns)
alpha = model.predict(X_current)
```

Lasso 自动剔除无效因子 (系数缩到 0)。风险: 容易过拟合, 必须 walk-forward 验证。

Thinking: 什么是walk-forward验证?

验证是时间序列专用的交叉验证方法:

用一段历史窗口 (如 2022–2024) 训练模型, 在紧接着的未来窗口 (如 2025Q1) 测试; 然后整体向前滚动一个步长 (如 2023–2025 训练, 2025Q2 测试), 重复多次。

=> 严格按时间顺序验证, 避免用未来信息预测过去

因子合成的方法

IC 与 IR 计算

```
def calc_ic(factor_series, future_return, method="spearman"):
    # Spearman 秩相关 -- 不受极值影响, 最常用
    return factor_series.corr(future_return, method=method)

def calc_ir(ic_series):
    # IR = IC 均值 / IC 标准差
    return ic_series.mean() / ic_series.std()
```

分层回测 + 单因子 IC 诊断

分层回测原理

- 每个调仓日 (默认月度), 按 alpha 把所有股票分 5 层
- L1 = alpha 最低的 20%, L5 = alpha 最高的 20%
- 每层等权持有到下个调仓日
- 看 5 层的累计收益曲线: 如果层数越高收益越高, 因子有效
- 多空收益 = $L5 - L1$ = 因子的纯 alpha

多因子合成 alpha: 10 个因子 (动量/波动率/换手率...) 经 去极值 + Z-score + 行业中性化 后的 等权平均

CASE: 多因子选股

实测 -- 沪深 300 前 80 只 $\times$ 400 日 $\times$ 月度调仓 $\times$ 12 期

```
python layered_backtest.py --max-stocks 80 --lookback 400 --rebal 21 --ic-lookback 6
```

Thinking: 这里的参数代表什么?

12 期 = 12 次月度调仓

实际是程序里推算出来的调仓次数:

400 日数据 - 130 日 warmup - 21 日尾部 (要算下期收益) $\approx$ 249 日

249 日 / 21 日 $\approx$ 12 次

每次调仓就是一次 walk-forward 滚动: T 时刻只用 T 之前的数据算因子 + 用过去 6 期 IC 算权重 $\rightarrow$ 等权或 IC 加权合成 alpha $\rightarrow$ 选股 $\rightarrow$ 持有 21 天 $\rightarrow$ 算下期收益 $\rightarrow$ 滑到 T+21 重复。所以 12 期 = 12 次"算 $\rightarrow$ 选 $\rightarrow$ 持 $\rightarrow$ 评分"循环。

CASE: 多因子选股

实测 -- 沪深 300 前 80 只 $\times$ 400 日 $\times$ 月度调仓 $\times$ 12 期

```
python layered_backtest.py --max-stocks 80 --lookback 400 --rebal 21 --ic-lookback 6
```

Thinking: 这里的参数代表什么?

--ic-lookback 6 是另一个东西: 每次合成因子时回看过去 6 期 (6 个月) 的 IC 估权重。

400 日 = 拉多少根日 K 线

--lookback 400 让 xtdata 拉 400 个交易日的 K 线 $\approx$ 1.5 年历史。

其中前 130 日纯热身 (因子要 warmup, 比如 MOM_6M 需要 120 日历史)

=> 真正参与回测的是后面的 270 日 $\approx$ 12 期。

CASE: 多因子选股

```
python layered_backtest.py --max-stocks 80 --lookback 400 --rebal 21 --ic-lookback 6
```

80 只 $\neq$ 月度买入 80 只

--max-stocks 80 是 股票池大小 -- 把沪深 300 砍成"前 80 只"作为候选 (节省时间, 全量 300 太慢), 不是每次买 80 只。

每月真正持有的股票数由代码里的 top_n_list=[5, 10, 20] 决定 -- 在 80 只候选里按 alpha 排序, 同时跑 3 套:

- Top-5 组合: 每月持有最高 alpha 的 5 只 (等权)
- Top-10 组合: 每月持有最高 alpha 的 10 只
- Top-20 组合: 每月持有最高 alpha 的 20 只

外加分层回测 (Q1/Q2/Q3/Q4/Q5), 把 80 只按 alpha 平均切 5 层各 16 只持有, 看最高分组 (Q1) 和 最低分组 (Q5) 的收益差 (= 多空对冲收益) => 用来检验因子有效性的核心指标。

CASE: 多因子选股

实测 -- 沪深 300 前 80 只 × 400 日 × 月度调仓 × 12 期

```
python layered_backtest.py --max-stocks 80 --lookback 400 --rebal 21 --ic-lookback 6
```

=> 在沪深 300 前 80 只里, 拉 1.5 年日 K, 每月初按多因子 alpha 排序,

分别买入 Top 5/10/20 只持有 1 个月再换仓, 总共换 12 次, 看年化收益和分层单调性

Thinking: 运行结果如何?

CASE：多因子选股

1) 单因子 IC 排名 -- 谁是真 alpha, 谁是噪音

ic_mean	ic_std	ic_ir	ic_positive_ratio	samples	
LIQ_20	0.0508	0.1358	0.3741	0.6667	12 ← 最强 (低流动性 alpha)
VOL_60	0.0316	0.2074	0.1525	0.5000	12
TURN_20	0.0155	0.1165	0.1331	0.5000	12
RSI_14	0.0148	0.1477	0.1003	0.5000	12
VOL_20	0.0060	0.1790	0.0336	0.5000	12
BIAS_20	-0.0027	0.1298	-0.0208	0.3333	12
_1M	-0.0136	0.1242	-0.1091	0.5833	12
REV_5D	-0.0338	0.0981	-0.3445	0.4167	12 ← 反向 (短期反转的反转)
_3M	-0.0467	0.1215	-0.3843	0.3333	12
_6M	-0.0556	0.1110	-0.5008	0.2500	12 ← 反向最强 (大盘股反转)

洞察 1: 沪深 300 的反转效应远比想象顽固。

_6M / _3M / REV_5D 的 IR 全部为负, 而且 _6M 达到 -0.50 -- 这是 IR 强度排第二的因子 (绝对值)!

=> 过去 6 个月涨得越多的大盘股, 下个月跌得越狠。

我们之前做过全市场 6900 只里的"动量 IR 较弱"形成鲜明对比 -- 中小盘有动量效应, 大盘股是反转, 完全相反。

=> 因子是有适用域的, 同一个动量, 在不同股票池的 IR 可以从 +0.2 翻到 -0.5。

CASE: 多因子选股

2) 等权 vs IC 加权 -- 用同样的因子, 让模型自动给反向因子负权重

Thinking: 既然 5 个因子是反向 (IR 为负), 5 个是正向, 等权合成 (10 个因子直接相加 / 10) 必然互相抵消。怎么办?

解法: IC 加权 -- 用过去 6 期的单因子 IC 滚动均值作为权重 (walk-forward, 不偷看未来),
反向因子自动得到负权重 => 相当于自动反转。

```
# layered_backtest.py 的核心一段
if weight_method == "ic_weighted":
    # 用过去 ic_lookback 期的 IC 滚动均值, 严格 walk-forward (不含当期)
    past_ic = {col: np.mean(history[col][-6:]) for col in factors}
    alpha = ic_weighted_synthesis(factor_processed, past_ic)
else:
    alpha = factor_processed.mean(axis=1) # 等权基线
```

CASE：多因子选股

指标	等权合成	IC 加权	变化	
IC 均值	-0.0164	+0.0008	+0.0172	← 从负翻正
IC IR	-0.0963	+0.0063	+0.1026	← 方向救活
IC 正比例	66.7%	50.0%	-16.7%	
L5 累计收益	+12.90%	+8.66%	-4.24%	
L1 累计收益	+9.49%	+21.15%	+11.66%	
L5-L1 多空收益	+3.41%	-12.48%	-15.90%	← 看似变差?
Top10 累计收益	+10.55%	-1.59%	-12.14%	
Top10 最大回撤	-5.85%	-9.38%	-3.52%	

洞察 2: IC 加权救活了 IC 均值, 但没救活策略

这是真实回测里最常见的反直觉

Thinking: IC 均值从 -0.016 翻到 +0.001, IR 从 -0.10 提到 +0.01 -- 数学上方向修正成功了。但 L5-L1 反而从 +3% 跌到 -12%, 为什么?

=> walk-forward 的滞后性。过去 6 期的 IC 估的是已经发生过的反转, 但市场的反转 / 动量切换不是均匀的, 等模型用历史 IC 把反向因子权重压负, 当期市场可能已经在切换回动量市了。

结论: IC 加权修正方向有效, 但稳定收益还要叠加 (a) 更长 lookback 平滑权重 (b) 因子去噪 (剔除 VOL_20/BIAS_20 这种 IR≈0 的) (c) 半衰期加权 (近期 IC 权重更大)。

CASE：多因子选股

3) Top-N 集中度 -- 越集中越好? 沪深 300 上不一定

分层回测告诉我们因子方向, 但实盘要回答到底买几只。

这里跑 Top-5 / Top-10 / Top-20 三档, 看集中度的边际效应。

[等权合成]				[IC 加权]			
组合	累计	每期均值	最大回撤	组合	累计	每期均值	最大回撤
Top5	-2.79%	-0.18%	-5.74%	Top5	-3.53%	-0.14%	-15.12%
Top10	+10.55%	+0.91%	-5.85%	Top10	-1.59%	-0.04%	-9.38%
Top20	+10.51%	+0.94%	-6.55%	Top20	+11.19%	+0.99%	-4.44%

CASE: 多因子选股

洞察 3: 沪深 300 上越集中越输, 之前中小盘Top-10 最佳完全相反。

等权: Top-5 亏 2.8%, Top-10/20 都赚 ~10.5%

IC 加权: Top-5/10 亏, Top-20 才赚 11.2%

原因: 大盘股之间收益率分散度小 (个股年化差距远小于中小盘), 因子的边际信息有限。

集中持有最看好的 5 只 在中小盘里意味着集中获取小市值溢价 (Top-10 多因子打分赚 18.88%);

但在大盘股里只是"押对几个名字 vs 押错几个名字"的运气比拼, 自然要分散到 20 只才能稳定。

第10节课里: 小市值 Top-20 反而比 Top-10 收益高

=> 这跟本 CASE 沪深 300 上Top-20 才稳是同一个原理:

因子越接近群体效应 (低流动性 / 小市值), 越分散越稳;

因子越接近个股质量 (基本面 / 财务), 越集中越好。

CASE: 多因子选股

4) 沪深 300 基准对照 -- 没有基准的回测都是耍流氓

沪深 300 同期累计收益 (12 个月): +13.43%

我们的策略 vs 基准:

等权 Top10: +10.55% → 跑输基准 -2.88%

等权 Top20: +10.51% → 跑输基准 -2.92%

IC 加权 Top20: +11.19% → 跑输基准 -2.24%

IC 加权 L1: +21.15% → 跑赢基准 +7.72% (反向选股)

CASE: 多因子选股

洞察 4: 这套技术因子在沪深 300 上, 没有任何按 alpha 顺方向的组合跑赢基准。

唯一跑赢的是 L1 (alpha 最低 20% 等权) +21.15%。这意味着什么?

- 技术因子的 alpha 在大盘股上几乎被市场吃光了。

沪深 300 是聪明钱密度最高的池子, 单纯的动量 / 波动率 / 流动性早被定价。

要在这里挖 alpha, 必须叠加财务因子 (ROE / 营收增速 / 现金流) 或事件因子 (财报 / 北上资金流入)。

Summary

反直觉发现	对应数据	背后原理
沪深 300 是反转市	_6M IR=-0.50, _3M IR=-0.38, REV_5D IR=-0.34	大盘股交易拥挤, 动量被市场提前定价 → 涨多必跌
等权合成 IC $\approx$ 0	IC=-0.016, IR=-0.10	5 个反向因子和 5 个正向因子直接相加互相抵消
IC 加权救方向不救收益	IC 翻正但 L5-L1 反而恶化	walk-forward 滞后, 反转切换时权重跟不上
沪深 300 上越集中越输	Top-5 亏, Top-20 才稳	大盘股分散度小, 因子边际信息有限, 集中没意义

CASE: 多因子选股

Thinking: 下一步可以做什么?

- 换股票池: 中证 1000 / 全市场过滤 ST 上重测, 验证动量在中小盘是正向假说
- 剔除噪音因子: VOL_20 / BIAS_20 / RSI_14 IR 接近 0, 实盘可直接剔除, 留 LIQ_20 + VOL_60 + _6M (反向使用) 三个
- IC 加权改进: lookback 拉长到 12 期 + 半衰期加权 (近期 IC 权重大), 缓解滞后
- 叠加财务因子: ROE / 营收增速 / 现金流, 用 tushare / wind / 自建财务库补

这是大盘股池子里唯一未被技术因子吃掉的 alpha 源

- 结合交易成本: 月度换 80 只 $\rightarrow$ 1 只, 印花税 + 佣金 + 滑点扣 0.3% / 月, 看年化是否还正

Summary

虽然没有找到特别适合的策略，但是建立了一个能客观评估策略好坏的工具:

- 单因子 IC 帮你筛掉噪音因子
- 等权 vs IC 加权告诉你权重方法的边界
- Top-N 集中度告诉你应该买几只
- 基准对照告诉你这个策略到底有没有 alpha

打卡：多因子分层回测



以沪深 300 前 80 只 x 400 日 x 月度调仓 x IC 加权为例

- 给每只股票算 10 个因子 (/REV/VOL/LIQ/TURN/RSI/BIAS), 全部统一成 => 越大越好 方向。
- 数据处理三件套 -- MAD 去极值 + Z-score 标准化 + 行业内中性化
- 用过去 6 期单因子 IC 滚动均值做权重 (walk-forward 不偷看未来), 反向因子自动得到负权重。
- 月度按 alpha 分 5 层, 看 L5-L1 多空收益; 同时输出 Top-5/10/20 集中度对照 + 沪深 300 基准。
- 整理分层回测结果, 并进行洞察

板块轮动分析

CASE：板块数据准备

板块数据准备：

只要你想做板块层面的任何分析, 就绕不开这一步。常见场景有 3 类:

场景	需要的板块数据	示例
板块强度排名	板块每日涨跌幅 + 成交额	今天 AI / 新能源 / 消费谁更强
板块指数 K 线	板块每日 OHLC + 成交量	光伏设备过去 60 天均线斜率怎样
个股归因	股票 → 板块的映射	贵州茅台属于哪个二级板块, 同板块还有谁

问题在于: xtdata / akshare 这些数据源直接给的是成分股清单 + 个股 K 线, 不是合成好的板块指数。
所以这里我们需要:

- 申万二级 134 个板块 × 每个板块的 N 只成分股 → 等权合成 OHLC 板块指数 → 落地到 MySQL
- 可以直接 SQL 一行读出某个板块的指数, 不需要每次重新从成分股算起。

CASE：板块数据准备

- 申万一级 vs 申万二级

维度	申万一级	申万二级
数量 (2021 版)	31 个	134 个
例子	食品饮料 / 电力设备 / 医药生物 / 电子	白酒 / 光伏设备 / 创新药 / 半导体
适合场景	宏观风格判断, 大类资产配置	板块轮动, 细分赛道挖掘
信号灵敏度	偏滞后 (一级是聚合, 头部股票拉动效应弱)	更敏感 (二级里的 5-30 只标的更同质)
典型市场风格	全球宏观策略 / 风格因子	行业主题轮动 / 题材爆发追踪

这里主要使用：申万二级 (134 个)

=> 板块轮动信号更敏感, 一年回测出来的拐点事件也更多。

CASE：板块数据准备

- 3 张 MySQL 表把个股 K 线 → 板块归类 → 板块指数完整建模

上下游各 CASE 都直接 SQL 读这三张表:

表名	主键	关键字段	本 CASE 干的事
trade_stock_status	stock_code	stock_name, list_date, sector_1, sector_2	从 xtdata 同步申万一级/二级分类, 写入 sector_1 / sector_2
trade_stock_daily	(stock_code, trade_date)	open / high / low / close, volume, amount	从 xtdata 增量下载个股日 K 线, 上溯到 SW_INIT_START_DATE
trade_sector_daily	(sector_name, sector_level, trade_date)	open_idx / high_idx / low_idx / close_idx, change_pct, total_volume, total_amount, kline_stock_count	每天从 trade_stock_daily 聚合成分股, 累乘合成板块指数 OHLC

- 申万一级和二级共存一张表 -- trade_sector_daily.sector_level 字段区分, 一个板块就是一行, 不需要建两张表
- 主键去重 + ON DUPLICATE KEY UPDATE -- 增量跑可以反复执行, 不会写脏数据
- open_idx / high_idx / low_idx / close_idx 四列齐全 -- 这样后续可以算 MACD / MA / 真实波幅 等技术指标

CASE: 板块数据准备

- 板块指数合成口径 (累乘法)

板块指数是合成的, 不是原生的, 算法决定了它跟"申万官方指数"是否吻合。

这里用业内行业指数最主流的累乘法:

起始基期: $\text{close_idx0} = 1000$

递推: $\text{close_idx}_t = \text{close_idx}_{t-1} \times (1 + \text{mean}(\text{close_ret}_i, t) \text{ for } i \text{ in members})$

其中: $\text{close_ret}_{i,t} = \text{close}_{i,t} / \text{close}_{i,t-1} - 1$

Thinking: 为什么用累乘而不是 close 等权均值?

- 对停牌/退市鲁棒: 前一日 close_idx 已经吸收了, 不会因成员减少出现跳空
- 业内行业指数主流口径: 中证、申万官方指数都用类似累乘逻辑 (流通市值加权 + 累乘)
- 增量更新友好: 每天只算 成份股均收益 就够, 不需要回看历史

CASE: 板块数据准备

```
def aggregate_sector_one_day(sector_name, sector_level, member_codes,
                             trade_date, prev_trade_date,
                             prev_close_idx, kline_panel):
    close_rets, open_rets, high_rets, low_rets = [], [], [], []
    for code in member_codes:
        today = kline_panel.get((code, trade_date))
        prev = kline_panel.get((code, prev_trade_date))
        if not today or not prev or not prev["close"]:
            continue
        close_rets.append(today["close"] / prev["close"] - 1)
        open_rets.append (today["open"] / prev["close"] - 1)
        high_rets.append (today["high"] / prev["close"] - 1)
        low_rets.append (today["low"] / prev["close"] - 1)
```

```
close_idx = prev_close_idx * (1 + np.mean(close_rets))
open_idx  = prev_close_idx * (1 + np.mean(open_rets))
high_idx  = prev_close_idx * (1 + np.mean(high_rets))
low_idx   = prev_close_idx * (1 + np.mean(low_rets))
# ... 写入 trade_sector_daily, ON DUPLICATE KEY UPDATE
```

运行结果

全量初始化 (只跑一次, 30-90 分钟, 取决于网络 and xtdata 速度)

cd CASE-A-板块数据准备 && pip install -r requirements.txt

cp .env.example .env # 配 MySQL 连接 + 起始日期 SW_INIT_START_DATE

python run_init.py # 建表 + 同步申万分类 + 个股 K 线 + 合成板块指数

后续每日 8:30 cron 触发增量 (秒级到分钟级)

python run_daily.py

运行结果

以起始日 2024-01-01 → 截止日 2025-12-31 两年数据为例, 跑完 CASE-A 后数据库三张表的结果:

trade_stock_status : ~5,400 行 (全 A 股 + 申万一级/二级映射)

trade_stock_daily : ~2,500,000 行 (5,400 只 × ~485 个交易日)

trade_sector_daily : ~80,000 行 (134 二级 + 31 一级 = 165 板块 × 485 日)

运行结果

洞察 1: 申万二级板块 2024-2025 涨跌幅排行

SQL 一行就能查出某个时间窗口内, 134 个二级板块的
累计涨幅:

```
SELECT sector_name,  
       ROUND(MAX(CASE WHEN trade_date = '2025-12-31' THEN close_idx END) /  
             MAX(CASE WHEN trade_date = '2024-01-02' THEN close_idx END) - 1, 4) AS  
ret_2y  
FROM   trade_sector_daily  
WHERE  sector_level = 2  
GROUP BY sector_name  
ORDER BY ret_2y DESC  
LIMIT 10;
```

2024-01-02 → 2025-12-31 申万二级 Top 5 涨幅板块

sector_name	ret_2y	
半导体	+147.3%	← AI 算力主升, 寒武纪 / 中芯国际 领涨
通信设备	+112.5%	← 海外 AI 资本开支拉动, 中际旭创 / 新易盛
光模块	+108.2%	
游戏	+89.4%	← AI 应用落地预期 + 版号政策松绑
小金属	+73.6%	← 锂 / 稀土供给侧

2024-01-02 → 2025-12-31 申万二级 Bottom 5 跌幅板块

sector_name	ret_2y	
房地产开发	-38.2%	← 销售持续探底
航空机场	-29.5%	
教育	-25.1%	
燃气	-18.7%	
酒店餐饮	-15.3%	

运行结果

洞察 2: 单板块 close_idx 走势 -- 拐点一目了然

选半导体 这个 2024 年最猛的二级板块, 拉它的 close_idx 季度抽样:

```
SELECT trade_date, close_idx, change_pct, kline_stock_count
FROM trade_sector_daily
WHERE sector_name = '半导体' AND sector_level = 2
AND trade_date IN ('2024-01-02', '2024-04-01', '2024-07-01',
                    '2024-10-08', '2025-01-02', '2025-12-31');
```

半导体 close_idx 季度抽样				
trade_date	close_idx	change_pct	kline_stock_count	
2024-01-02	1000.00	0.00	96	
2024-04-01	1085.40	+1.20%	96	
2024-07-01	1042.30	-0.85%	96	
2024-10-08	1456.70	+9.85%	97	← "924 行情"启动日, 单日+9.85%
2025-01-02	1768.20	+0.45%	98	
2025-12-31	2473.50	+1.20%	98	← 两年累计 +147.3%

Summary

- 板块指数 `close_idx` 是可比的

不管成分股怎么变, 同一基期同一算法, 不同板块直接比涨幅

- 单日 `change_pct` 异常 (e.g. +9.85%) = 政策/事件冲击

2024-09-24 央行降准降息一揽子政策, 半导体作为高 beta 板块单日暴涨,

=> 这种事件信号在分析里非常珍贵

- `kline_stock_count` 漂移

96 → 98 说明这两年新增了 2 只半导体上市公司, 代码可以捕捉到

打卡：板块数据准备



以申万二级板块、起始日 2024-01-01 为例,

一键建表 + 同步申万分类 + 拉个股 K 线 + 合成 134 个二级板块指数:

- 建表 (sql/schema.sql): 一次性创建 `trade_stock_status` / `trade_stock_daily` / `trade_sector_daily`
- 从申万官网同步一级 + 二级行业元数据, 写入 `trade_stock_status.sector_1/sector_2`。
- 用 `xtdata` 拉全 A 股 ~5400 只前复权日 K, 落到 `trade_stock_daily` (一年约 250 万行)。
- 用累乘法 (基期 1000) 把成分股日 K 合成 165 个板块的 OHLC 指数, 落到 `trade_sector_daily`。

CASE：板块轮动分析

3层信号体系

3层信号体系:

- 静态强度 (零阶)

作用: 现在谁强

实现方式: industry_strength.py: $_21 + RS_60 + VOL_RATIO$ 三因子 z-score 合成

- 一阶导 (速度)

作用: 还在加速 / 减速

实现方式: derivatives.py: $ROC_20 + MA20_SLOPE$

- 二阶导 (加速度)

作用: 主升 / 见顶 / 见底 / 主跌

实现方式: derivatives.py: $MACD_HIST + MA20_ACCEL$

3层信号体系

Thinking: 为什么要二阶导?

把板块指数想象成价格质点:

- 价格本身 = 位置
- 价格变化率 (ROC, MA 斜率) = 速度, 一阶导
- 速度的变化率 (MACD 柱体, 斜率的变化) = 加速度, 二阶导

场景	速度	加速度	含义
主升加速 (accel_up)	正	正	越涨越快, 趋势延续, 强烈推荐持有
高位钝化 (decel_up)	正	负	还在涨但越涨越慢, 警惕见顶, 减仓信号
主跌 (accel_down)	负	负	越跌越快, 规避
左侧抄底 (decel_down)	负	正	还在跌但越跌越慢, 关注见底信号

5 个核心导数指标

5 个核心导数指标

指标	阶	公式	判读
ROC_20	1	$(close_t / close_{t-20} - 1) \times 100$	20 日涨幅, 短期速度
MA20_SLOPE	1	$(MA20_t - MA20_{t-5}) / MA20_{t-5} \times 100$	均线斜率, 平滑后的速度
MACD_HIST	2	$(DIF - DEA) \times 2$, 其中 $DIF = EMA12 - EMA26$	柱体高度, 加速度的经典代理
HIST_DELTA	2	$MACD_HIST_t - MACD_HIST_{t-1}$	柱体变化, 加速度的变化
MA20_ACCEL	2	$slope_now - slope_prev$	MA20 斜率的变化, 加速度

拐点判定逻辑

- 拐点判定逻辑

```
def detect_phase(d: Dict[str, float]) -> Dict[str, str]:  
    velocity_up = (d["ROC_20"] > 0.5 and d["MA20_SLOPE"] > 0.1)  
    velocity_dn = (d["ROC_20"] < -0.5 and d["MA20_SLOPE"] < -0.1)  
    accel_up   = (d["MACD_HIST"] > 0 and d["HIST_DELTA"] > 0  
                  and d["MA20_ACCEL"] > 0)  
    accel_dn   = (d["MACD_HIST"] < 0 and d["HIST_DELTA"] < 0  
                  and d["MA20_ACCEL"] < 0)  
  
    if velocity_up and accel_up: phase = "accel_up"    # 主升加速  
    elif velocity_up and accel_dn: phase = "decel_up"  # 高位钝化  
    elif velocity_dn and accel_dn: phase = "accel_down" # 主跌  
    elif velocity_dn and accel_up: phase = "decel_down" # 左侧抄底  
    else: phase = "neutral"
```

- **phase** 代表当前市场所处的趋势运动阶段,
- 通过价格速度 (velocity) 与加速度 (accel) 的组合, 识别主升、主跌、高位钝化或左侧抄底等状态

=> 用于判断趋势强度与潜在转折点。

综合视图

- 综合视图 (run_today.py)

把静态强度 z-score 和 phase 推荐分 加权合成, 得到当日综合排名:

$$\text{composite_score} = (_21_z + \text{RS_60_z} + 0.5 \times \text{VOL_R_z}) + \text{PHASE_BONUS}$$

其中 PHASE_BONUS: accel_up=+3, decel_down=+2, decel_up=+0.5, accel_down=-2, neutral=0

```
python run_today.py --level 2 --top 10
```

输出 4 份文件给晨会 workflow 消费:

- sector_combined_today.csv: 完整数据
- sector_strength_today.csv: 静态强度单表
- sector_phase_today.csv: phase 单表
- sector_today_topN.txt: 人类可读 Top 10 摘要 (晨报会嵌这个)

一年滚动回测

运行: CASE-B-板块轮动分析/backtest.py => 输出落到 outputs/rotation_events.csv

回测逻辑:

- 给每个板块, 滚动算每天的 phase
- 检测 phase 切换事件 (e.g. 从 neutral → accel_up)
- 评估事件后 5/10/20 日的板块绝对收益 + 相对市场基准的超额收益
- 统计 Top 10 最强切换事件 (按 excess_20d 倒序)

一年滚动回测

```
python backtest.py --lookback 252 --forward 20 --level 2
```

[BT] 板块轮动回测 (level=2, 回看 252 日)

[BT] 有效板块: 130

[BT] 总事件数: 11,436

[BT] phase 分布:

to_phase count

neutral 5221

accel_up 2338

decel_up 1569

accel_down 1449

decel_down 859

最近一年 Top 10 强势轮动事件 (按 excess_20d):

#1 贵金属 2025-12-26 neutral -> accel_up excess_20d=+97.76% abs_20d=+102.08%

#2 贵金属 2026-01-05 neutral -> accel_up excess_20d=+92.29% abs_20d=+95.84%

#3 影视院线 2025-11-18 neutral -> accel_up excess_20d=+58.15% abs_20d=+49.24%

#4 贵金属 2025-12-22 neutral -> accel_up excess_20d=+56.52% abs_20d=+59.79%

#5 航天装备 2025-12-04 neutral -> accel_up excess_20d=+52.85% abs_20d=+48.90%

#6 航天装备 2025-12-15 neutral -> accel_up excess_20d=+52.49% abs_20d=+55.01%

#7 地面兵装 2025-05-30 neutral -> accel_up excess_20d=+51.24% abs_20d=+52.19%

#8 贵金属 2026-01-13 neutral -> accel_up excess_20d=+46.62% abs_20d=+52.12%

#9 林业 2025-10-27 neutral -> accel_up excess_20d=+45.95% abs_20d=+38.58%

#10 航天装备 2025-12-24 neutral -> accel_up excess_20d=+39.04% abs_20d=+44.97%

一年滚动回测

仅看 Top 10 这一份榜单，可以看出来的结论：

- Top 10 全是 neutral → accel_up

中性盘整后突然进入主升加速 是能打的拐点信号, 不一定是 decel_down (左侧抄底) → accel_up

- 贵金属一个板块占了 4 席 (#1/#2/#4/#8)

这是 2025-12 至 2026-01 黄金大牛市的真实印记, 一旦某个板块进入趋势, 它会反复触发同方向信号

- 军工三兄弟 (航天装备/地面兵装) 占 4 席

政策驱动板块的拐点信号特别清晰, 因为基本面催化非常集中

Thinking: 但是只看 Top 10 不够, 11,436 个事件里到底有什么规律?

打卡：当日板块轮动



以申万二级 134 个板块、当日 Top 10 为例, 输出晨会要消费的 4 份文件:

- 算 134 个板块的 $_21 + RS_60 + VOL_RATIO$ 三因子, 各自 z-score 后相加得静态强度分。
- 算导数指标: $ROC_20 + MA20_SLOPE$ (一阶导, 速度) + $MACD_HIST + MA20_ACCEL$ (二阶导, 加速度)。
- 用速度方向 x 加速度方向把每个板块分到 4 象限 (主升加速 / 高位钝化 / 主跌 / 左侧抄底)。
- 合成综合分: $composite = \text{强度 z-score} + PHASE_BONUS$ (accel_up=+3, decel_down=+2, decel_up=+0.5, accel_down=-2)。
- 得到 4 份文件: 完整 csv + 强度单表 + phase 单表 + 人类可读 Top 10 摘要 (晨会 workflow 直接读取)。

事件规律挖掘

再深挖一层: rotation_insights.py 从 11,436 事件里挖出的 4 类规律

backtest.py: 只输出原始事件清单 + Top 10。

rotation_insights.py 直接读 outputs/rotation_events.csv, 沿 4 个维度做统计

=> 让板块轮动从感觉变成数字。

```
python rotation_insights.py # 读 rotation_events.csv, 落 outputs/insights/
```

事件规律挖掘：切换矩阵

规律 1: 切换矩阵 -- 16 种 from → to 路径, 哪几条最赚?

所有 11,436 个 phase 切换事件按 (from_phase, to_phase) 二维分组, 看每条路径的"事件数 / 平均后 20 日超额收益 / 胜率":

[1] 切换矩阵 Top 5 (按平均后 20 日超额收益)

from_phase	to_phase	count	mean_ex20d	win_rate	
accel_up	accel_down	11	+4.94%	63.6%	← 主升后过山车: 急涨急跌的均值回归交易窗
accel_down	accel_up	13	+4.61%	91.7%	← V 型反转: 91.7% 胜率最高, 但样本最少
accel_down	decel_down	49	+3.89%	75.0%	← 真正的"左侧抄底" -- 主跌中减速, 是信号最稳的进场点
decel_down	neutral	537	+2.04%	67.0%	← 砸完不再砸 = 风险出清, 后市自然反弹
neutral	decel_down	735	+1.59%	66.0%	

[1] 切换矩阵 Bottom 3 (最值得回避)

accel_up	decel_up	81	-0.80%	46.8%	← 主升钝化, 减仓信号
neutral	decel_up	1343	-1.05%	49.7%	
decel_up	neutral	1215	-1.37%	48.5%	← 高位回落到震荡, 真正的"接盘陷阱"

事件规律挖掘：切换矩阵

这张矩阵有几个反直觉:

- 以为最值钱的 `decel_down` → `accel_up` 其实没进 Top 5

⇒ 真实数据里它只有 +1.04% / 60% 胜率 (见衰减表), 和直觉差别很大。

原因: 直接从主跌减速跳到主升加速, 通常意味着情绪剧烈反转, 真正的右侧买点反而需要走到 `neutral` 中转一下

- 真正最稳的左侧信号是 `accel_down` → `decel_down` (+3.89% / 75% 胜率)

=> 主跌减速 是空头力量衰竭的最早证据, 比 已经反弹 更安全

- 最差的不是主跌, 而是 `decel_up` → `neutral` (-1.37% / 48.5%)

=> 主升钝化后退到中性, 是经典的高位接盘信号, 比 `accel_down` 还危险

事件规律挖掘：信号衰减曲线

规律 2: 信号衰减曲线 -- 是该立刻冲, 还是等几天?

同一条切换路径在 5d / 10d / 20d 三个时间窗口的平均超额收益对比:

[2] 信号衰减曲线 (count >= 30 的 8 条主路径)

path	5d	10d	20d	decay (20d-5d)
accel_down -> decel_down		+1.20%	+2.70%	+3.89% +2.69%
decel_down -> neutral		+0.28%	+1.19%	+2.04% +1.76%
neutral -> decel_down		+0.38%	+1.06%	+1.59% +1.21%
decel_down -> accel_down		-1.00%	+0.76%	+1.32% +2.32%
decel_down -> accel_up		-0.37%	-0.82%	+1.04% +1.41%
decel_up -> accel_up		-0.45%	+0.50%	+0.70% +1.15%
accel_down -> decel_up		+0.19%	-0.22%	+0.61% +0.42%
accel_down -> neutral		+0.32%	+0.19%	+0.49% +0.17%

- 几乎所有看多型切换在 5d 都是负的或微正
信号刚出来时市场还在消化, 立刻追涨胜率不高。20d 才是信号
的真正释放窗口
- 看多信号普遍呈"负 → 正"的 J 形曲线 (e.g. decel_down -> accel_up: -0.37% → -0.82% → +1.04%)
第二周往往是最难熬的下杀, 但 20 天后才是兑现。这就是经典的"右侧坚定, 左侧痛苦"
- 反向解读: 短线交易者 (持有 5d) 应该选 accel_down -> decel_down (5d 就有 +1.2%), 而不是直觉的 decel_down -> accel_up (5d 是 -0.37%)。不同持有期适合不同信号

事件规律挖掘：板块信号可靠度排行

规律 3: 板块信号可靠度排行 -- 哪些板块的拐点信号更有效?

对每个板块单独算 accel_up + decel_down 看多信号的平均超额收益和胜率, 按胜率最高排行:

[3] 板块信号可靠度 Top 10 (信号至少 3 次)

sector	#signals	mean_ex20d	win_rate	best	worst	
贵金属	20	+20.05%	75.0%	+97.76%	-19.36%	← 2025-12
黄金牛市, 拉爆均值						
航天装备	21	+6.58%	52.4%	+52.85%	-17.91%	
广告营销	20	+6.06%	60.0%	+24.69%	-7.68%	
游戏	25	+5.56%	88.0%	+14.07%	-17.60%	← 胜率最高, 信号最干净
装修装饰	16	+5.30%	75.0%	+15.57%	-4.36%	
油服工程	23	+4.60%	78.3%	+19.38%	-3.67%	
IT服务	19	+4.57%	78.9%	+12.35%	-5.51%	
医疗服务	22	+4.22%	86.4%	+12.40%	-18.42%	
军工电子	23	+4.12%	65.2%	+23.60%	-15.86%	

[3] 信号最不靠谱的 5 个板块

旅游零售	24	-4.20%	33.3%	+9.32%	-18.75%	
旅游及景区	16	-4.54%	43.8%	+13.86%	-33.28%	
饮料乳品	21	-4.75%	38.1%	+11.70%	-22.95%	
商用车	28	-5.00%	39.3%	+9.97%	-25.58%	
白酒	22	-8.58%	27.3%	+17.74%	-32.60%	← 信号"反指", 越说看多越跌

事件规律挖掘：板块信号可靠度排行

- 贵金属 / 军工 / 游戏 / 医疗服务 是 phase 信号的优质受体

这些行业要么趋势性强 (贵金属/军工), 要么基本面催化集中 (游戏/医疗服务), 拐点信号噪音小

- 白酒 / 饮料 / 旅游 / 商用车 是信号反指

这些都是 2024-2026 持续承压的消费板块, 趋势是"长期下台阶 + 短期反弹"。

accel_up 信号往往是"超跌反弹被诱多"的典型陷阱

- 胜率最高的不是赚最多的

游戏 88% 胜率 / 平均 +5.56%, 贵金属 75% 胜率 / 平均 +20.05%。

趋势型行业的信号波动大但赔率高, 震荡型行业的信号稳但赔率低

对于实战的建议：phase 信号 + 板块白名单, 给 贵金属/军工/游戏/医疗服务 加权重, 给 白酒/饮料/旅游/商用车 进行屏蔽 (信号反指)

事件规律挖掘：月度切换密度

规律 4: 月度切换密度 -- accel_up 减 accel_down 是免费的市场温度计

每个月统计 134 个二级板块发生了多少次 accel_up 切换, 多少次 accel_down 切换, 两者相减就是当月市场情绪净值:

[4] 月度切换密度 (近 12 个月)

month	accel_up	accel_down	bull-bear	市场情绪
2025-05	276	41	+235	普涨牛市
2025-06	172	112	+60	震荡偏多
2025-07	319	58	+261	主升爆发, 全市场启动
2025-08	233	33	+200	延续
2025-09	130	217	-87	普跌, 调整启动
2025-10	204	88	+116	反弹
2025-11	199	129	+70	震荡
2025-12	188	230	-42	年底跨年震荡
2026-01	289	47	+242	春季躁动 (黄金 + 军工领涨)

2026-02	71	81	-10	高位钝化
2026-03	73	277	-204	春季 correction, 全市场退潮
2026-04	133	87	+46	初步企稳

bull = 处于 accel_up (主升加速) 状态的板块数量;
bear = 处于 accel_down (主跌) 状态的板块数量。
=> 两者差值作为全市场多空温度计, 净多则暖、净空则风险。

事件规律挖掘：月度切换密度

- bull - bear = 能用的市场温度计

不需要任何宏观判断, 只看 130 个板块的拐点投票 => 净多则全市场偏暖, 净空则系统性风险来临

- 极端值 (± 200) 往往是行情拐点的滞后确认

2025-07: +261 => 牛市顶, 2026-03: -204 = 调整底。

用这个指标做 加杠杆/降仓位 的二档信号比看 K 线更冷静

- 跟我们的多因子选股策略联动:

bull-bear 高时 ($> +150$) 用 accel_up 信号 + 高仓位;

bull-bear 低时 (< -100) 改用 accel_down -> decel_down 左侧信号 + 半仓;

0 附近用震荡策略 (CASE-C 多因子选股而不是板块轮动)

Summary

以 130 个有效二级板块 x 252 日回看 x 20 日观察期为例, 验证拐点信号到底有没有用:

```
python backtest.py --lookback 252 --forward 20 --level 2 && python rotation_insights.py
```

先回测出全部 phase 切换事件, 再用挖掘器从 11K+ 事件里分析出有价值的规律:

- backtest.py: 给每个板块滚动算每日 phase, 检测切换事件 (e.g. neutral -> accel_up), 算事件后 5/10/20 日的板块 + 相对市场超额收益。
- 产出 outputs/rotation_events.csv (11,436 行) + Top 10 强势事件清单 (黄金、军工反复占榜, 印证信号共振)。
- rotation_insights.py 分析规律:

规律1: 16 种 from->to 路径切换矩阵 -- 找出最稳的左侧信号 accel_down -> decel_down (+3.89%, 75% 胜率)。

规律 2-3: 信号衰减 J 形曲线 (5d 下杀 / 20d 兑现) + 板块可靠度榜单 (游戏 88% 胜率最干净, 白酒 27.3% 胜率反指)。

规律 4: 月度 bull-bear 净值 = 免费市场温度计, +/-200 极值往往是大盘拐点的滞后确认。

CASE：晨会 workflow

CASE: 晨会 workflow

TO DO: 用 LangGraph 把 板块轮动 + 多因子选股 串成 4 节点 DAG

cron 9:00 自动跑一遍

场景: 你的AI量化基金, 每天 8:30 - 9:20 都会开投资晨会:

- **宏观 / 策略组:** 复盘隔夜外盘 + 国内政策 → 给出: 今天关注哪些板块
- **行业研究员:** 在强势板块里挖具体标的 → 给出: 今天关注哪些股票
- **会议纪要:** 形成晨报, 9:20 前推送到投研群 + 销售交易团队
- **9:30 开盘:** 基金经理参考晨报开始操作



CASE：晨会 workflows

维度	20-团队 workflow	21-晨会 workflow
形状	含循环的状态机 (Kris 否决 → 回到 Zoe 重试)	线性 DAG (4 节点串成一条线)
用到的 LangGraph 特性	5 件套全用: StateGraph + Conditional Edge + Interrupt + Checkpointer + Command	3 件套即可: StateGraph + Node + Edge
触发场景	盘中事件触发 (信号到了)	盘前定时触发 (cron 9:00)
处理对象	单标的全流程 (一笔交易)	全市场批量选股 (一份报告)
业内对照	类似 Temporal 的 决策状态机	类似 Airflow / Prefect 的 批处理 DAG

这里都统一使用的LangGraph, 只需掌握一套编排工具即可。

架构设计

节点职责

节点	职责	调用 (lib/)	State 输出
industry	申万二级板块强度 + 一二阶导拐点	rotation_runner. rank_industries_with_phase	industry_rank (含 phase + composite_score)
stock_picker	Top 板块成分股 → 多因子打分 → 选 Top N	rotation_runner. get_sector_member_codes + factor_runner. calc_factors_batch	stock_pool, factor_rank, picked_stocks
report	拼装晨报 Markdown + HTML 双格式	本节点内置 _md_to_html	report_md, report_html
push	推送钉钉 / 企业微信 / console	pusher.push_all	push_result

架构设计

CASE-D 的 lib/ 目录里, 没有 import CASE-B 或 CASE-C, 而是把核心逻辑各自复制了一份:

CASE-D-投资晨会 workflow/

└─ graph.py

└─ pusher.py

└─ lib/

| └─ db_config.py # 复制自 CASE-A

| └─ rotation_runner.py # 复制自 CASE-B (sector_loader + industry_strength + derivatives + run_today)

| └─ factor_runner.py # 复制自 CASE-C (factor_lib + preprocessor + stock_pool)

└─ README.md

State 设计

```
class MorningState(TypedDict, total=False):
```

```
    # 输入
```

```
    trigger_time: str
```

```
    industry_level: int # 申万级别 1 / 2 (默认 2)
```

```
    top_n_industries: int
```

```
    top_n_stocks: int
```

```
    lookback_days: int
```

```
    sample_stocks: int
```

```
    # 中间产出
```

```
    industry_rank: list # industry 节点写, 含 phase 字段
```

```
    stock_pool: list # stock_picker 节点写
```

```
    factor_rank: list
```

```
    picked_stocks: list
```

```
    # 输出
```

```
    report_md: str # report 节点写
```

```
    report_html: str
```

```
    push_result: dict # push 节点写
```

```
    # 审计日志 (Annotated reducer 让多节点可 append)
```

```
    messages: Annotated[list, add]
```


运行结果

```
python graph.py --level 2 --top-industries 5 --top-stocks 5 --sample-per-industry 20
```

[节点 1] industry_node -- 申万二级板块强度 + 一二阶导拐点

Top 5 板块 (申万 二 级):

- [1] 光伏设备 score=+5.42 (主升加速) MOM21=+12.30% RS60=+8.50% ROC20=+11.20%
- [2] 半导体 score=+4.18 (主升加速) MOM21=+9.40% RS60=+5.60% ROC20=+8.10%
- [3] 创新药 score=+3.32 (左侧抄底) MOM21=-2.10% RS60=-1.40% ROC20=-1.80%
- [4] 白酒 score=+2.85 (主升加速) MOM21=+6.10% RS60=+3.20% ROC20=+5.50%
- [5] 风电设备 score=+2.41 (主升加速) MOM21=+5.50% RS60=+2.80% ROC20=+4.90%

[节点 2] stock_picker_node -- Top 板块成分股做多因子选股

候选股票池: 87 只 (来自 5 个 Top 板块)

[FACTOR] 82 只有效, 5 只数据不足被剔除

Top 5 选中标的:

- 688472.SH [光伏设备] alpha=+0.385 _3M=+18.50%
- 002129.SZ [光伏设备] alpha=+0.342 _3M=+15.20%

运行结果

603501.SH [半导体] alpha=+0.318 _3M=+12.10%

603259.SH [创新药] alpha=+0.295 _3M=-3.40% ← 左侧抄底信号配低位标的

600519.SH [白酒] alpha=+0.273 _3M=+8.10%

[节点 3] report_node -- 拼装晨报

晨报已落盘:

Markdown: outputs/reports/morning_brief_20260424_085724.md

HTML: outputs/reports/morning_brief_20260424_085724.html

[节点 4] push_node -- 推送钉钉 / 企业微信

[推送] 投资晨会 04-24 (channel: console)

推送结果: {'console': True, 'dingtalk': False, 'wecom': False} # 没配 webhook

Summary

- 主升加速板块 + 高_3M 标的:

光伏设备前两位 (688472 / 002129) 都是基本面在改善 + 趋势在加速, 这是偏右侧的 momentum 选股

- 左侧抄地板块 + 低_3M 标的:

创新药 phase 是 decel_down (跌幅在收敛), 选出的 603259 _3M 还是 -3.4%, 这是偏左侧的反转选股

- 同一份策略覆盖了趋势和反转两类机会

这就是"板块 phase + 多因子打分"两层叠加的价值, 单纯多因子在熊市底部容易踩空, 单纯板块强度则容易追高

Summary

- 留意信号反指板块:

这次输出里白酒被选进 Top 5 板块, 但 之前的板块可靠度排行显示白酒 phase 信号 -8.58% / 胜率 27.3%, 是典型反指。

实战做法: 在 industry_node 后加一道白名单/黑名单过滤

=> 把 贵金属/军工/游戏/医疗服务 加权, 把 白酒/饮料/旅游/商用车 屏蔽

=> 不要让纯统计信号在已知反指板块上下注

定时调度

Thinking: 关于定时调度的 cron / apscheduler / Windows 任务计划程序 三个分别是什么?

名词	本质	能跨平台?
cron	Linux/macOS 系统自带的"定时任务守护进程", 配置在 <code>crontab</code>	不能 -- Linux/macOS 独有
APScheduler	Python 库, 自己起一个进程当调度器, 支持 cron 风格的时间表达式	能 (Windows / Linux / macOS 都行)
Windows 任务计划程序	Windows 系统自带的"定时任务面板" (taskschd.msc)	Windows 独有

我们用 APScheduler, 它跨平台, 在 Windows 下 `python scheduler.py` 一行就启动。

cron 在这里用来表达定时任务 (比如: 每个工作日 9:00), 不是 Linux 那个守护进程。

定时调度

核心就 BlockingScheduler + 两个 cron 触发器, 每个工作日 (mon-fri) 跑一次:

```
from apscheduler.schedulers.blocking import BlockingScheduler
from apscheduler.triggers.cron import CronTrigger

# Job 1: 每天 8:30 拉昨日增量数据 (调子进程)
def job_data_refresh():
    subprocess.run([sys.executable, "CASE-A-板块数据准备/run_daily.py"])

# Job 2: 每天 9:00 跑晨会 workflow (直接 import build_graph, 比开子进程快)
def job_pre_market():
    from graph import build_graph
    build_graph().invoke({"industry_level": 2,
                          "top_n_industries": 5,
                          "top_n_stocks": 10})

sched = BlockingScheduler(timezone="Asia/Shanghai")
sched.add_job(job_data_refresh, CronTrigger(hour=8, minute=30, day_of_week="mon-fri"))
sched.add_job(job_pre_market, CronTrigger(hour=9, minute=0, day_of_week="mon-fri"))
sched.start() # 阻塞前台运行, Ctrl+C 退出
```

CronTrigger(hour=9, minute=0,
day_of_week="mon-fri") 是cron 表达式
=> 只在工作日跑, 周末跳过, 不需要再写 if

timezone="Asia/Shanghai" 必填
=> 服务器装 Linux 默认 UTC 时区, 不写的话
9:00 会变成 17:00, 这是新手最容易踩的坑

BlockingScheduler.start() 是阻塞式的
=> 启动后 Python 进程不会退出, 你需要把它
放后台 (Linux: nohup ... &; Windows: 任务计划
程序设开机自启), 否则关掉终端调度器就停了

scheduler.py

定时调度

开始用之前, 别等到第二天 9:00 才发现哪里有 bug。

scheduler.py 内置 --simulate, 立刻把两个 job 各跑一次:

- `python scheduler.py --simulate # 立刻跑 8:30 + 9:00 两个 job`
- `python scheduler.py --simulate --job pre # 只跑 9:00 那个 (常用)`

simulate 跑完会立刻退出 (不进入阻塞), 看到 [JOB] job_pre_market 完成, 推送结果: ...

=> 说明编排没问题

晨会 workflow

早上 8:30 - 9:30 时间线

- 08:30 cron 触发 CASE-A run_daily, 增量更新昨日个股 K 线 + 当日 165 板块指数
- 08:35 增量完成 (秒级到分钟级, 取决于网络)
- 09:00 cron 触发 CASE-D LangGraph workflow
- 09:05 industry_node 完成 134 板块强度排名 + phase 检测
- 09:15 stock_picker_node 完成多因子选股, 得到 Top 5 标的
- 09:18 report_node 拼装 Markdown + HTML 晨报
- 09:20 push_node 推送钉钉, 手机收到晨报
- 09:30 开盘, 你已经看完晨会, 知道今天关注什么

打卡：晨会 workflow



以申万二级 Top 5 板块 + 每板块 Top 5 股 + 每天 9:00 自动跑为例，打造晨会 workflow

- 排出 Top 5 强势板块 + phase 标签。
- 在 Top 5 板块成分股里算 10 因子 + 三件套预处理, 取 Top N。
- 拼装 Markdown / HTML 晨报落到 outputs/reports/, 调 pusher.py 推送钉钉/企业微信群。
- Windows 直接 python scheduler.py 跑前台



Thank You
Using data to solve problems

